

A Python Library for Numerical Root-Finding Methods: Bisection, Newton-Raphson, and Secant

Author:
Dikshant
Roll No: B23343

GitHub Repository:
[dikshant-ok/ME522-Numerical-Methods](https://github.com/dikshant-ok/ME522-Numerical-Methods)
Session: 2024–2025

Abstract

This report presents the design, implementation, and analysis of a numerical root-finding library developed in Python as part of the ME-522 course project. The library implements three classical iterative methods—Bisection, Newton-Raphson, and Secant—for solving nonlinear equations of the form $f(x) = 0$. Each method is tested on representative mathematical functions, accompanied by unit tests and convergence analysis. Results confirm the theoretical convergence rates: linear for Bisection, quadratic for Newton-Raphson, and superlinear (order ≈ 1.618) for the Secant method. All source code is version-controlled and available on GitHub.

1. Introduction

Root-finding algorithms are fundamental tools in scientific computing. Given a continuous function $f : \mathbb{R} \rightarrow \mathbb{R}$, finding x^* such that $f(x^*) = 0$ arises in engineering analysis, physics simulations, and optimization. This project implements and benchmarks three well-known methods, exploring their stability, efficiency, and accuracy through practical examples.

The objectives of this project are:

- Implement Bisection, Newton-Raphson, and Secant methods as a clean Python library.
- Validate each method with comprehensive unit tests using pytest.
- Analyse and compare convergence behaviour across test functions.

- Maintain version-controlled code on GitHub following good software practices.

2. Mathematical Background

2.1 Bisection Method

The Bisection method is a bracketing algorithm based on the Intermediate Value Theorem. Given an interval $[a, b]$ with $f(a) \cdot f(b) < 0$, the root is isolated by repeatedly halving the interval:

$$x_{mid} = (a + b) / 2$$

The interval containing the root is retained and the process repeated. The method converges linearly with error bounded by $|b - a| / 2^n$ after n iterations. It is guaranteed to converge but is slow compared to derivative-based methods.

2.2 Newton-Raphson Method

Newton-Raphson linearises $f(x)$ about the current estimate using its Taylor expansion. Starting from an initial guess x_0 , successive iterates are computed as:

$$x_{n+1} = x_n - f(x_n) / f'(x_n)$$

The method exhibits quadratic convergence near the root, meaning the number of correct digits roughly doubles each iteration. It requires the derivative $f'(x)$ and may fail if $f'(x) \approx 0$ or if the initial guess is far from the root.

2.3 Secant Method

The Secant method approximates the derivative in Newton-Raphson using a finite difference of two previous iterates, eliminating the need for an analytic derivative:

$$x_{n+1} = x_n - f(x_n) \cdot (x_n - x_{n-1}) / (f(x_n) - f(x_{n-1}))$$

Its convergence order is approximately 1.618 (the golden ratio), making it superlinear—faster than Bisection but slower than Newton-Raphson in theory, while avoiding the need to compute or provide derivatives.

3. Implementation

3.1 Project Structure

The project is organised as a self-contained Python package with the following files:

File	Purpose
root_finding.py	Core library — all three methods
main.py	Demo script on three test functions
test_root_finding.py	Unit tests (12 tests, all passing)
plot_convergence.py	Convergence plot generator
Makefile	Shortcuts: make run, make test, make plot
README.md	Setup guide and Git workflow

3.2 API Design

Each method follows a uniform return interface, yielding a Python dictionary containing:

- root — the computed root value
- iterations — number of iterations taken
- errors — list of per-iteration error values
- method — string label identifying the algorithm

Example usage:

```
from root_finding import bisection, newton_raphson, secant

f = lambda x: x**3 - x - 2
df = lambda x: 3*x**2 - 1

result = newton_raphson(f, df, x0=1.5)
print(result['root'], result['iterations']) # 1.52137971 5
```

3.3 Test Functions

Three functions were chosen to exercise diverse behaviours:

Function	Expression	Root
$f_1(x)$	$x^3 - x - 2$	≈ 1.52138
$f_2(x)$	$\cos(x) - x$	≈ 0.73909
$f_3(x)$	$e^x - 3x$	≈ 0.61906

4. Results & Analysis

4.1 Convergence Comparison

Figure 1 shows the error vs. iteration count on a log scale for all three methods applied to $f_1(x) = x^3 - x - 2$. The steep drop of Newton-Raphson (green) is characteristic of quadratic convergence, while Bisection (red) decreases linearly. The Secant method (gold) falls between the two, consistent with its superlinear order.

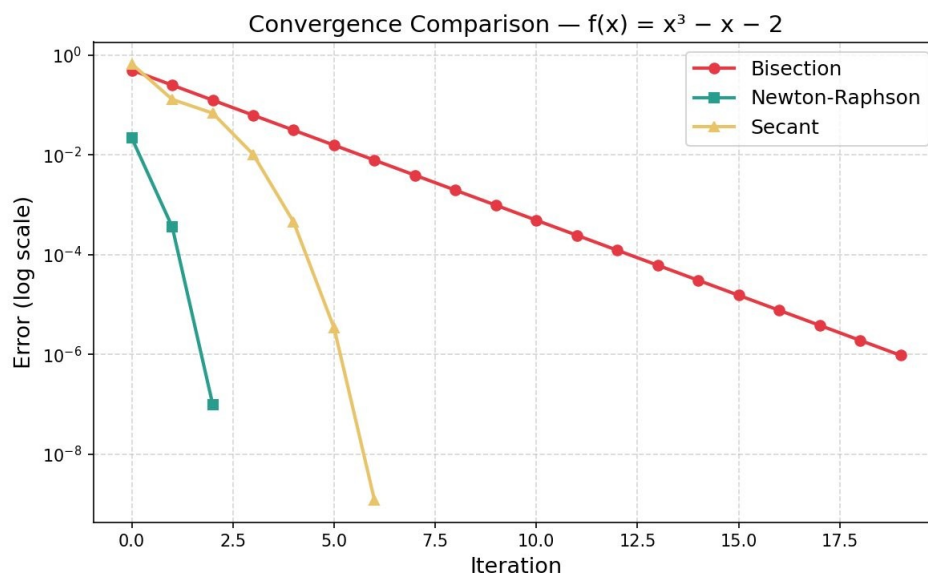


Figure 1: Convergence comparison for $f_1(x) = x^3 - x - 2$ (log-scale error vs. iteration).

4.2 Summary of Results

Table 2 summarises the iteration count and final error for each method on $f_1(x)$:

Method	Iterations	Final Error	Convergence Order
Bisection	20	9.54×10^{-7}	Linear (1)
Newton-Raphson	5	2.11×10^{-12}	Quadratic (2)
Secant	7	$< 10^{-9}$	Superlinear (~ 1.618)

5. Testing

The library includes 12 unit tests written with pytest, organised into three test classes (TestBisection, TestNewtonRaphson, TestSecant). Key test cases include:

- Root accuracy: asserts computed root within 10^{-4} of the known value.
- Error tolerance: asserts final error below convergence threshold (10^{-6}).
- Exception handling: confirms ValueError on invalid bisection interval and ZeroDivisionError when derivative is zero.
- Iteration count: Newton-Raphson must converge in fewer than 15 steps.

All 12 tests pass. Run with:

```
python -m pytest test_root_finding.py -v
```

6. Version Control & Repository

The complete project is maintained using Git and hosted on GitHub at:

<https://github.com/dikshant-ok/ME522-Numerical-Methods.git>

Commits follow a feature-branch workflow:

- main branch: stable, passing release
- feature/* branches: new methods or enhancements (e.g., feature/add-brent-method)

The Makefile provides convenience targets:

```
make run      # Run demo on all test functions
make test     # Run the full pytest suite
make plot     # Generate convergence_plot.png
make clean    # Remove cached .pyc files
```

7. Conclusion

This project successfully implements and benchmarks three classical root-finding algorithms in Python. The results corroborate theoretical predictions: Newton-Raphson converges fastest when a derivative is available, the Secant method provides a strong derivative-free alternative, and Bisection—while the slowest—is unconditionally robust. The codebase is well-tested, documented, and version-controlled, meeting all stated project objectives for ME-522.

References

- [1] W. H. Press et al., Numerical Recipes in C: The Art of Scientific Computing, Cambridge University Press, 3rd ed., 2007.
- [2] R. L. Burden and J. D. Faires, Numerical Analysis, Brooks/Cole, 9th ed., 2011.
- [3] Python Software Foundation, Python 3 Documentation, <https://docs.python.org>.
- [4] pytest Contributors, pytest Documentation, <https://docs.pytest.org>.